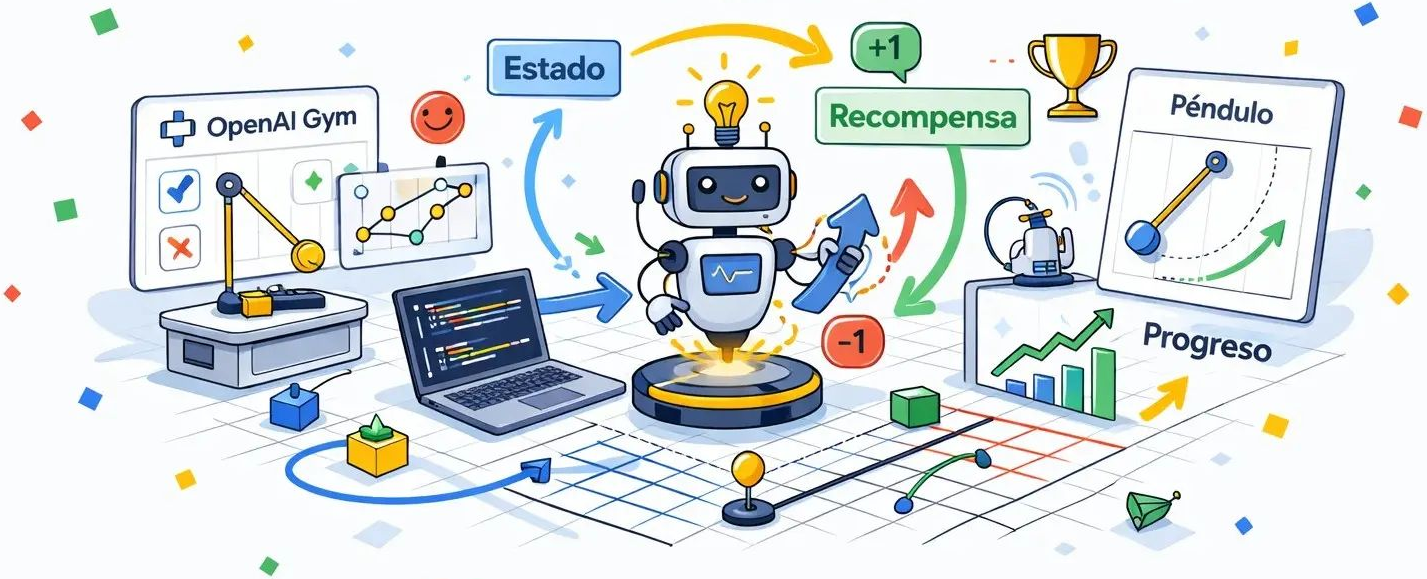
DÍA 2 · REINFORCEMENT LEARNING

Aprendizaje por recompensas, OpenAI Gym y control de un péndulo



Google Developer Group
Universidad Autónoma de Madrid

¿QUÉ ES EL REINFORCEMENT LEARNING?

El paradigma

Agente interactuando con su entorno

El ciclo

Agente recibe recompensas o castigos

Los elementos clave

agente, entorno y política

CONCEPTOS BÁSICOS

Agente

Toma la decisión

Entorno

Donde ocurre la acción

Estado

Situación actual del entorno

Acción

Decisión que toma el agente

Recompensa

Feedback que indica que tan buena es la acción

OpenAI Gym

Gymnasium

¿Qué es?

librería para definir y ejecutar entornos de aprendizaje por refuerzo de forma estandarizada.

¿Qué tiene?

Proporciona entornos propios, que se usan como estándares para el aprendizaje automático

Entornos

CartPole, MountainCar, Acrobot, Pendulum...

Uso

Permite probar agentes en entornos controlados

RECURSOS Y CÓDIGO DEL EVENTO

Todo el material, ejemplos y el código está disponible aquí

github.com/Hugo3ΘB/gym-rl

OpenAI Gym

Entorno	Tipo de acción	Dificultad	Problema principal	Algoritmo usado	Reward esperada
CartPole-v1	Discreta	Baja	Mantener el equilibrio	PPO	Alta/Episodio largo
MountainCar-v0	Discreta	Media-Alta	Llegar a la cima con una estrategia útil	PPO	Baja al inicio, va subiendo
Acrobot-v1	Discreta	Media	Coordinar movimiento y balanceo	PPO	Mejora progresiva
Pendulum-v1	Continua	Media-Alta	Control estable del sistema	PPO	Menos negativa cuanto mejor estabiliza
Bala2Env	Continua	Alta	Equilibrio robusto	PPO	Positiva y estable si mantiene equilibrio

Stable-Baselines 3

¿Qué es?

Librería de aprendizaje por refuerzo basada en PyTorch

Propósito

Entrenar agentes sin implementar algoritmos desde cero

Integración

Se utiliza junto a entornos de Gymnasium para interactuar y aprender

Algoritmos

Incluye PPO, DQN, A2C, SAC y TD3, entre otros

Ventaja clave

Facilita experimentación rápida con código simple y reproducible

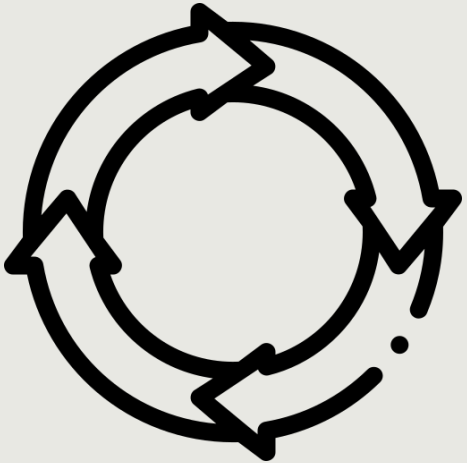
Algoritmos RL

Política

Estrategia que decide qué acción tomar en cada estado

Aprendizaje

Se actualiza la política para mejorar decisiones futuras



Interacción

El agente actúa en el entorno y observa resultados

Evaluación

Se calcula la recompensa obtenida (qué tan buena fue la acción)

Proximal Policy Optimization

¿Qué es?

Algoritmo que optimiza directamente la política de forma estable

Idea clave

Actualiza la política en pequeños pasos para evitar cambios bruscos

Como lo hace

Limita cuánto puede cambiar la política en cada actualización

Objetivo

Mejorar el rendimiento sin perder estabilidad durante el entrenamiento

```python
env = gym.make(environment_name,render_mode="human")
```

```python
episodes = 5
for episode in range(1, episodes+1):
    state, info = env.reset()
    done = False
    score = 0

    while not done:
        action = env.action_space.sample()
        n_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

        score += reward

    print(f'Episode:{episode} Score:{score}')
```

```python
env.reset()
```

```python
env.step(1)
```

```python
env.action_space
```

```python
env.action_space.sample()
```

```python
env.observation_space
```

```python
env.observation_space.sample()
```

```python
env.close()
```

```python
# Veamos la estructura del entorno
env = gym.make("CartPole-v1")
print(f"Espacio de observacion: {env.observation_space}")
print(f"  -> Forma: {env.observation_space.shape}")
print(f"  -> Limites: [{env.observation_space.low}] a [{env.observation_space.high}]")
print(f"\nEspacio de acciones: {env.action_space}")
print(f"  -> Numero de acciones: {env.action_space.n}")
print(f"\nEjemplo de observacion inicial:")
obs, info = env.reset()
print(f"  -> {obs}")
print(f"  -> [posicion_carro, velocidad_carro, angulo_palo, velocidad_angular_palo]")
env.close()
```

```
Espacio de observacion: Box([-4.8               -inf -0.41887903        -inf], [4.8               inf 0.41887903        inf], (4,), float32)
  -> Forma: (4,)
  -> Limites: [[-4.8               -inf -0.41887903        -inf]] a [[4.8               inf 0.41887903        inf]]

Espacio de acciones: Discrete(2)
  -> Numero de acciones: 2

Ejemplo de observacion inicial:
  -> [-0.04871993 -0.00934357  0.01575027 -0.00837166]
  -> [posicion_carro, velocidad_carro, angulo_palo, velocidad_angular_palo]
```

```python
# Agente aleatorio - nuestra linea base
frames_random_cartpole = record_episode("CartPole-v1", model=None)
print(f"El agente aleatorio sobrevivio {len(frames_random_cartpole)} pasos (de 500 posibles)")
show_animation(frames_random_cartpole)
```

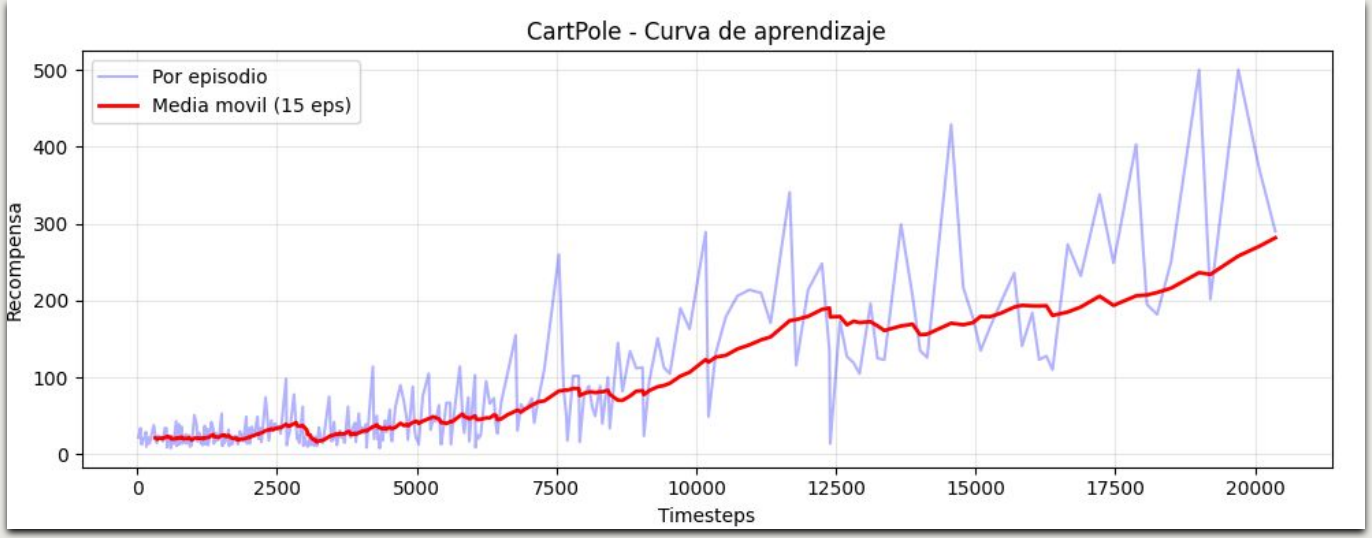
```
El agente aleatorio sobrevivio 24 pasos (de 500 posibles)
```

```python
# Creamos el entorno con Monitor para registrar las recompensas
log_dir = "logs/cartpole"
os.makedirs(log_dir, exist_ok=True)

env = Monitor(gym.make("CartPole-v1"), log_dir)
env = DummyVecEnv([lambda: env])

# Creamos el modelo PPO
model_cartpole = PPO(
    "MlpPolicy",       # Red neuronal fully-connected (Multi-Layer Perceptron)
    env,
    device=device,
    verbose=1,
    learning_rate=3e-4, # Tasa de aprendizaje (default de PPO)
    n_steps=2048,       # Pasos antes de cada actualizacion
    batch_size=64,      # Tamano del mini-batch
)

print("Arquitectura de la red neuronal del agente:")
print(model_cartpole.policy)
```



Sim-to-Real: Robot Bala2

Entorno y Algoritmo

- **Bala2Env:** Entorno Gymnasium diseñado para simular las físicas del robot equilibrista.
- **Stack:** Python + Gymnasium + Stable Baselines 3.
- **PPO:** Algoritmo de optimización que permite actualizaciones estables de la política.
- **Objetivo:** Lograr un equilibrio en simulación para exportarlo al hardware real.

Flujo Sim-to-Real: De la simulación en Gymnasium al hardware real en tiempo real.

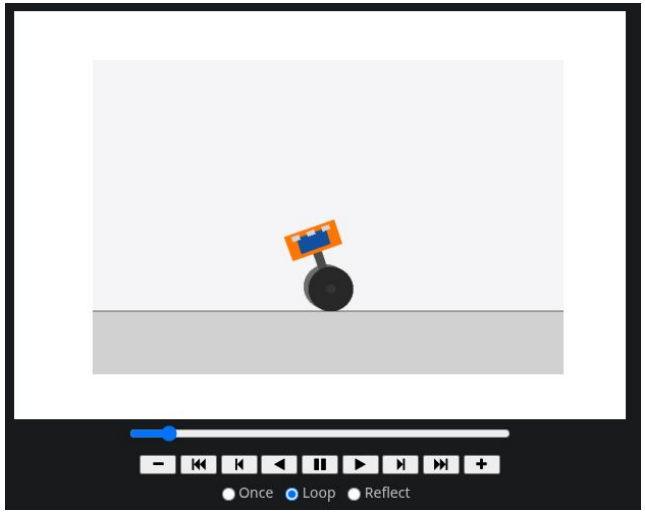
Entorno Gymnasium: Bala2Env

Especificaciones Físicas

- **Masa Total:** ~0.285 kg (Chasis + Cuerpo)
- **Control:** Frecuencia de 200 Hz (tau=0.005s)
- **Límites:** ±40° de inclinación, ±2.4m de pista
- **Modelo:** Dinámica de cuerpo rígido con Inercia

Sim-to-Real: Realismo Crítico

- **Motor:** Simulación de lag y zona muerta
- **Ruido:** Jitter y encoders en observaciones
- **Robustez:** Domain Randomization (masas y fuerzas)
- **Penalización:** Castigo a los tirones



Representación del robot en Pygame. El agente aprende a equilibrar el robot aplicando fuerzas laterales compensatorias.

Usamos MLPPolicy: Multi-Layer Perceptron (Perceptrón Multicapa) que recibe Vectores

Diseño de la Función de Recompensa

$$\text{Reward} = 1.0 - [\ 2.0*(\theta/\theta_lim)^2 + 0.1*(x/x_lim)^2 + 0.1*(\theta)^2 + 0.1*(cmd)^2\ ]$$

Base: 1.0

Incentivo constante por cada paso de tiempo que el robot sobrevive sin caer.

Ángulo (θ)

Penalización crítica (x2). Castigo cuadrático por desviarse de la vertical.

Posición (x)

Mantiene el robot en el centro. Penaliza la deriva lateral en la pista.

Suavidad (cmd)

Evita comandos bruscos y oscilaciones para proteger el hardware real.

FALLO CRÍTICO: Reward = -10.0 (Si el robot cae o sale de límites)

Entrenamiento y Arquitectura PPO

1. Red Neuronal (MlpPolicy)

Optimización para ESP32

- **Arquitectura:** MLP (Multi-Layer Perceptron)
- **Capas:** 2 capas ocultas de 32 neuronas (net_arch=[32, 32])
- **Activación:** Tanh para transiciones suaves
- **Entrada:** Sensores de estado (x, $\dot{x}$, θ, $\dot{\theta}$)
- **Salida:** Acción de control para el motor [-1, 1]

2. Configuración PPO

```
// Hiperparámetros de Entrenamiento

model = PPO("MlpPolicy", env,
   learning_rate=3e-4,
   n_steps=2048,
   batch_size=128,
   n_epochs=10,
   gamma=0.999,
   gae_lambda=0.95,
   clip_range=0.2,
   policy_kwargs=dict(
      net_arch=[dict(pi=[32,32], vf=[32,32])]
   )
)


// Ejecución
model.learn(total_timesteps=100_000)
```

Flujo de Exportación: PyTorch a C++ (.h)

Lógica de Extracción Capa a Capa

El script de Python accede al **mlp_extractor** para procesar cada componente de la red neuronal:

- **Capa 1:** Extrae pesos `layer1_w` y sesgos `layer1_b` (32 neuronas).
- **Capa 2:** Extrae parámetros de `policy_net[2]`.
- **Salida:** Procesa el `action_net` para obtener la política de control final.

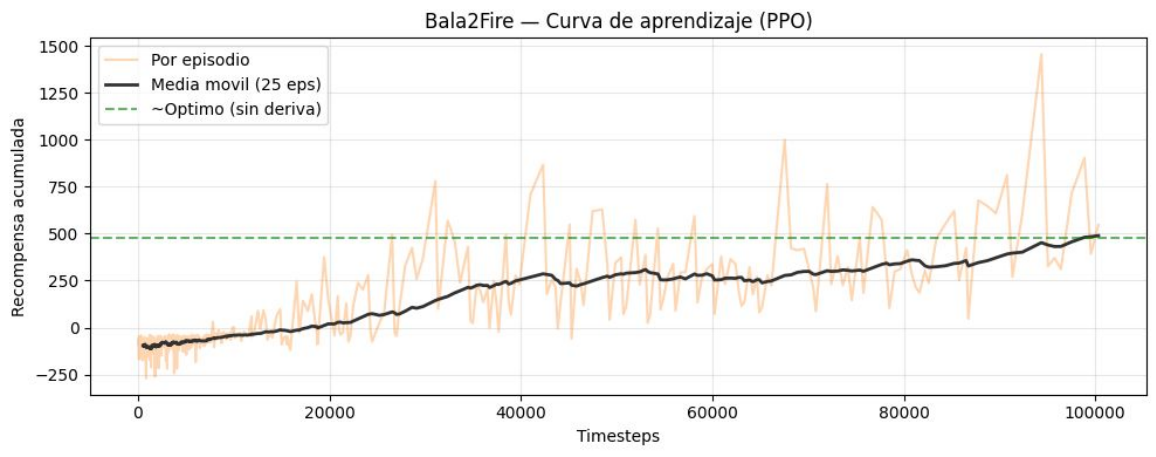
CONTENIDO GENERADO EN model_data.h

```
// Capa 1: [32 x 4]
const float layer1_w[32][4] = { ... };
const float layer1_b[32] = { ... };

// Capa 2: [32 x 32]
const float layer2_w[32][32] = { ... };
const float layer2_b[32] = { ... };

// Capa Salida: [1 x 32]
const float output_w[1][32] = { ... };
const float output_b = { ... };
```

Progreso del Entrenamiento (PPO)



Análisis del Aprendizaje

Convergencia: La media móvil (línea negra) tiene un crecimiento sostenido de la recompensa acumulada conforme avanzan los timesteps.

Resultado Final: Tras 100,000 pasos, el modelo supera la línea de referencia (verde), indicando que el robot ha aprendido a mantenerse en equilibrio de forma consistente.

Inferencia en Tiempo Real: Arduino C++

```
// Implementación en update_nn_motor_speed
float input[4] = {x, x_dot, theta, theta_dot};

// CAPA 1: Input (4) -> Hidden 1 (32)
for(int i=0; i<32; i++){
h1[i] = layer1_b[i];
for(int j=0; j<4; j++) h1[i] += input[j]*layer1_w[i][j];
h1[i] = tanhf(h1[i]);
}

// CAPA 2: Hidden 1 (32) -> Hidden 2 (32)
for(int i=0; i<32; i++){
h2[i] = layer2_b[i];
for(int j=0; j<32; j++) h2[i] += h1[j]*layer2_w[i][j];
h2[i] = tanhf(h2[i]);
}

// CAPA DE SALIDA: Hidden 2 (32) -> Action (1)
float action = output_b[0];
for (int j = 0; j < 32; j++) {
// output_w debe ser un array [1][32]
action += h2[j] * output_w[0][j];
}


// SALIDA Y ESCALADO PWM
action = clamp(action, -1.0, 1.0);
int16_t pwm = (int16_t)(action * 1023.0f);
```

PASO 1

Entrada de Sensores

Se extraen los estados físicos (x, θ...) en un vector de entrada de 4 dimensiones.

PASO 2

Forward Pass (Tanh)

Cálculo matricial capa a capa aplicando la activación tangente hiperbólica.

PASO 3

Saturación (Clamp)

Se limita la acción a [-1.0, 1.0] para asegurar consistencia con el entrenamiento.

FINAL

Mapeo a PWM

Conversión a rango [-1023, 1023] para el control directo de los motores del robot.

Sim-to-Real: Desafíos del Robot

Masa y Pesos

La distribución asimétrica del centro de gravedad genera muchas complicaciones para pasar de simulación a realidad.

Motores

Los límites de torque y la zona muerta de los motores deben modelarse con precisión para evitar oscilaciones, que es lo que nos ha ocurrido.

Energía

La caída de tensión por nivel de batería afecta a la potencia disponible.

Sensores

El ruido de alta frecuencia y la latencia introducen desfases críticos que el agente debe aprender a filtrar o anticipar.

Agarre

La fricción inconsistente superficie-rueda es el factor más difícil de simular.

CPU

La limitación de ciclo de cálculo en el Esp32 obliga a optimizar la red neuronal.

Simplificación 2D: **Motores sincronizados para emular el modelo matemático de Cart-Pole (un solo eje de libertad).**

RL en Videojuegos: Stable-Retro

Entrenando en Entornos de videojuegos retro

El Reinforcement Learning no se limita al hardware físico; los videojuegos son perfectos para algoritmos de IA.

`stable-retro` es un fork de OpenAI Retro que permite convertir miles de juegos clásicos a entornos de Gymnasium.

- Soporte para +1000 juegos de consola (NES, SNES, Genesis, etc).
- Acceso directo a la memoria RAM para extraer estados.
- Se puede subir casi cualquier ROM clásico y se convertirá automáticamente a un entorno gym

Gymnasium Retro y Stable Baselines 3 con Airstriker-Genesis-v0

Arquitectura de Stable-Retro

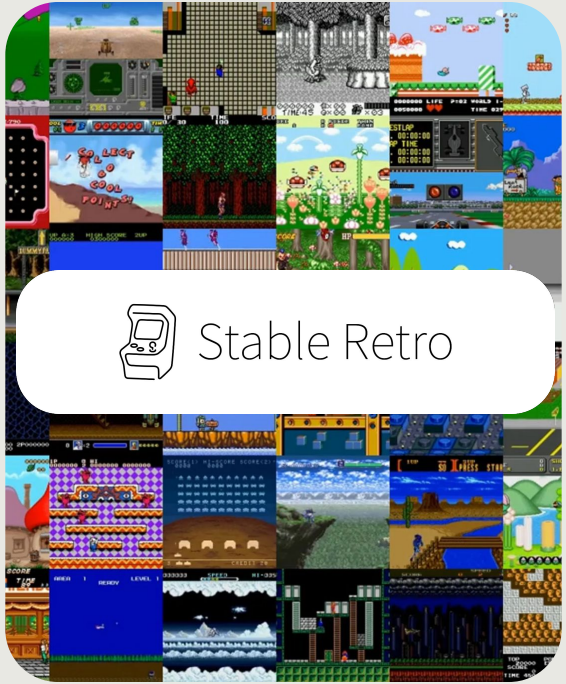
Entorno y Gestión

Stable Retro

Emulador de consolas clásicas con interfaz Gym: Envía la imagen (píxeles) y la recompensa (memoria RAM) a la IA.

DummyVecEnv

Wrapper obligatorio que permite que los algoritmos (como PPO) gestionen el flujo de datos de forma eficiente.



Procesamiento y Visión

CnnPolicy

Los "ojos" de la IA; red neuronal que analiza píxeles para detectar formas, enemigos y proyectiles.

VecFrameStack

La "memoria de movimiento" que apila los últimos 4 fotogramas para que el modelo pueda entender el moviemento.

¡Muchas Gracias!

- GitHub: github.com/GDG-UAM
- Contacto: hugo@bellido.net
- Contacto: pablo.derobledo@gmail.com